

NLP Lab 5: Transformer Code Walkthrough

Block 1: Environment & Seed Setup

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import math
import random
import itertools

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
torch.manual_seed(42)
random.seed(42)
```

- `import ...`: Loads PyTorch neural network modules (`nn`), optimization algorithms (`optim`), and data handling utilities (`DataLoader`, `TensorDataset`).
- `device = ...`: Automatically detects if a GPU (`cuda`) is available to accelerate matrix multiplications; otherwise, defaults to `cpu`.
- `torch.manual_seed(42)`: Locks the random number generator so weight initialization and data shuffling yield the exact same results every time you run the script.

Block 2: Toy Corpus Generation

```
pos_adjs = ["amazing", "brilliant", "fantastic", "great", "wonderful", "superb"]
pos_nouns = ["movie", "film", "story", "acting", "cinema", "masterpiece"]
pos_endings = ["loved it", "highly recommend", "a joy to watch", "perfect", "enjoyed every minute"]

# ... (negative lists omitted for brevity) ...

pos_sentences = [f"{a} {n} {e}" for a, n, e in itertools.product(pos_adjs, pos_nouns, pos_endings)]
pos_sentences = random.sample(pos_sentences, 600)

sentences = pos_sentences + neg_sentences
labels = [1] * 600 + [0] * 600 # 1: Positive, 0: Negative

combined = list(zip(sentences, labels))
random.shuffle(combined)
sentences, labels = zip(*combined)
```

- `itertools.product(...)`: Generates every possible combination of adjective + noun + ending to create thousands of synthetic reviews.
- `random.sample(..., 600)`: Randomly selects exactly 600 positive and 600 negative reviews to create a perfectly balanced dataset of 1,200 samples.
- `labels = ...`: Creates a corresponding list of integer targets (1 for positive, 0 for negative).
- `random.shuffle(...)`: Mixes the dataset so the model doesn't see all positive reviews first and all negative reviews second.

Block 3: Vocabulary and Tokenization

```
vocab = {"<PAD>": 0, "<UNK>": 1}
for sent in sentences:
    for word in sent.split():
        if word not in vocab:
            vocab[word] = len(vocab)

MAX_LEN = 8
def encode(sent):
    tokens = [vocab.get(w, vocab["<UNK>"]) for w in sent.split()]
    return tokens[:MAX_LEN] + [vocab["<PAD>"]] * max(0, MAX_LEN - len(tokens))

X = torch.tensor([encode(s) for s in sentences], dtype=torch.long)
y = torch.tensor(labels, dtype=torch.float32)

dataset = TensorDataset(X, y)
train_loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

- `vocab = ...`: A dictionary mapping words to unique integer IDs. Initializes `<PAD>` for filling short sentences and `<UNK>` for unknown words.
- `encode(sent)`: Converts a text string into a list of integers. Truncates lists longer than `MAX_LEN` (8) and appends `<PAD>` (0) to shorter ones.
- `torch.tensor(...)`: Converts Python lists into PyTorch tensors. Features (X) are long (integers) for embedding lookups; labels (y) are float32 for binary loss calculation.
- `DataLoader(...)`: Wraps the dataset into an iterator that serves 32 samples at a time (`batch_size=32`).

Block 4: Positional Encoding Module

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
```

```

        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0)
/ d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        return x + self.pe[:, :x.size(1), :]

```

- `torch.zeros(...)`: Initializes an empty matrix to hold the positional signals.
- `torch.sin / torch.cos`: Applies alternating sine and cosine waves of different frequencies to generate unique positional signatures for every token index.
- `self.register_buffer('pe', pe)`: Saves the `pe` tensor as part of the model's state so it moves to the GPU, but prevents the optimizer from trying to update it.
- `x + self.pe[...]`: In the forward pass, adds the positional wave values directly to the token embeddings.

Block 5: Transformer Classifier Architecture

```

class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, d_model=128, nhead=4, num_layers=2, num_classes=1,
max_len=32):
        super().__init__()
        self.d_model = d_model

        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=0)
        self.pos_encoder = PositionalEncoding(d_model, max_len=max_len)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, dim_feedforward=256, batch_first=True
        )
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)
        self.classifier = nn.Linear(d_model, num_classes)

    def forward(self, x, padding_mask=None):
        x = self.embedding(x) * math.sqrt(self.d_model)
        x = self.pos_encoder(x)

        encoded = self.transformer_encoder(x, src_key_padding_mask=padding_mask)
        pooled = encoded.mean(dim=1)
        logits = self.classifier(pooled).squeeze(-1)
        return logits

```

- `nn.Embedding`: A lookup table converting integer token IDs into dense vectors of size `d_model`. `padding_idx=0` ensures the <PAD> token always outputs zeros.
- `nn.TransformerEncoderLayer`: A single Transformer block containing Multi-Head Self-Attention and a FeedForward network. `batch_first=True` expects inputs as (Batch, Sequence, Features).
- `nn.TransformerEncoder`: Stacks multiple `encoder_layer` blocks sequentially.
- `x * math.sqrt(self.d_model)`: Scales embeddings up before adding positional encodings.
- `encoded.mean(dim=1)`: Mean Pooling. Averages the contextualized token vectors across the sequence length to compress the sentence into a single 1D vector.
- `squeeze(-1)`: Removes the trailing dimension from the classifier output to match the 1D target label shape.

Block 6: Training Loop

```
model = TransformerClassifier(vocab_size=len(vocab), d_model=64, nhead=2, num_layers=2,
                             max_len=MAX_LEN).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.BCEWithLogitsLoss()

model.train()
for epoch in range(5):
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        padding_mask = (inputs == 0).to(device)

        optimizer.zero_grad()
        logits = model(inputs, padding_mask=padding_mask)
        loss = criterion(logits, targets)
        loss.backward()
        optimizer.step()
```

- `optim.Adam`: The optimization algorithm that updates network weights based on computed gradients.
- `BCEWithLogitsLoss`: Binary Cross Entropy loss. It automatically applies a Sigmoid activation to the raw output logits.
- `padding_mask = (inputs == 0)`: Creates a boolean tensor where True marks a <PAD> token. Passed to the Transformer so self-attention ignores padding words.
- `optimizer.zero_grad()`: Clears leftover gradients from the previous batch to prevent accumulation.
- `loss.backward()`: Calculates the derivative (gradient) of the loss with respect to every weight in the network.
- `optimizer.step()`: Adjusts the weights in the direction that minimizes the loss.

Block 7: Inference Function

```
def predict(text, model, vocab, max_len=MAX_LEN):
    model.eval()
    tokens = [vocab.get(w, vocab["<UNK>"]) for w in text.split()]
    padded = tokens[:max_len] + [vocab["<PAD>"]] * max(0, max_len - len(tokens))

    input_tensor = torch.tensor([padded], dtype=torch.long).to(device)
    pad_mask = (input_tensor == 0).to(device)

    with torch.no_grad():
        logit = model(input_tensor, padding_mask=pad_mask)
        prob = torch.sigmoid(logit).item()
```

- `model.eval()`: Switches the model to evaluation mode, disabling training-specific behaviors like Dropout.
- `with torch.no_grad()`: Suspends PyTorch's gradient tracking engine, saving significant memory and compute time during inference.
- `torch.sigmoid(logit).item()`: Converts the raw logit score into a probability between 0.0 and 1.0, extracting it from the PyTorch tensor as a standard Python float.

Transformer Tensor Shapes & Data Flow

Understanding the dimensional transformations of data as it passes through the Encoder-Only Transformer. The shapes assume the following hyperparameters based on our toy corpus training:

- B = Batch Size (e.g., 32)
- T = Sequence Length / MAX_LEN (e.g., 8)
- D = Embedding Dimension / d_model (e.g., 64)
- C = Number of Classes (1 for Binary Classification)

Shape Transformation Summary

Processing Stage	Tensor Shape	Description
1. Raw Input Tokens	(B, T) -> (32, 8)	Integer indices mapped from the vocabulary.
2. nn.Embedding	(B, T, D) -> (32, 8, 64)	Each integer is mapped to a dense vector of size D.
3. Positional Encoding	(B, T, D) -> (32, 8, 64)	Sine/Cosine wave values are added; shape remains

		identical.
4. Transformer Encoder	(B, T, D) -> (32, 8, 64)	Self-attention processes context. Input and output shapes match exactly.
5. Mean Pooling	(B, D) -> (32, 64)	Averages across the Sequence dimension (T) to create a single vector per sentence.
6. Linear Classifier	(B, C) -> (32, 1)	Projects the D-dimensional feature vector down to a single logit per sentence.
7. Squeeze	(B,) -> (32,)	Removes the trailing dimension to match the 1D target label tensor.

Detailed Stage-by-Stage Breakdown

1. Input Data & Masking

- Input Tensor: $x = \text{inputs}$

Shape: (Batch_Size, Max_Length) -> (32, 8)

- Padding Mask: $\text{padding_mask} = (\text{inputs} == 0)$

Shape: (Batch_Size, Max_Length) -> (32, 8)

Type: Boolean matrix used to hide <PAD> tokens from the self-attention mechanism.

2. Embedding & Positional Injection

- Operation: $x = \text{self.embedding}(x) * \text{math.sqrt}(\text{self.d_model})$

Shape: (32, 8) becomes (32, 8, 64)

- Operation: $x = \text{self.pos_encoder}(x)$

Shape: Remains (32, 8, 64). Positional encoding matrix of shape (1, 8, 64) is broadcast-added to the embeddings.

3. Transformer Encoder Block

- Operation: $\text{encoded} = \text{self.transformer_encoder}(x, \text{src_key_padding_mask}=\text{padding_mask})$

Shape: Remains (32, 8, 64)

Context: Inside the encoder, Multi-Head Attention projects this into Queries, Keys, and Values of shape (32, 8, 64), splits them into 2 heads of shape (32, 2, 8, 32), computes attention, and concatenates back to (32, 8, 64).

4. Mean Pooling

- Operation: `pooled = encoded.mean(dim=1)`

Shape: (32, 8, 64) is collapsed to (32, 64)

Context: We must reduce the time sequence (8 tokens) into a single aggregate representation so the Linear layer can classify the sentence as a whole.

5. Classification Head & Loss

- Operation: `logits = self.classifier(pooled)`

Shape: (32, 64) becomes (32, 1)

- Operation: `logits = logits.squeeze(-1)`

Shape: (32, 1) becomes (32,)

Context: Matches the `y_target` shape (32,) so `BCEWithLogitsLoss` can compare them directly.